Trabajo Práctico 3 – Sistemas de Computación

Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

1 **Introducción** **3**

2 **MBR mínimo** **4**

2.1 Construcción del MBR 4

2.2 Prueba de arranque 4

3 **UEFI y Coreboot** **6**

3.1 UEFI 6

3.2 Uso de UEFI 6

3.3 Casos de bugs de UEFI 6

3.4 Converged Security and Management Engine (CSME) y Intel MEBx 7

3.5 Coreboot 7

4 **Linker** **9**

4.1 Dirección 0x7c00 y su importancia 9

4.2 Comparativa de objdump y hd 10

4.3 Opción --oformat binary en el linker 13

5 **Depuración con GDB Dashboard** **14**

6 **Modo protegido** **17**

6.1 Programa con dos descriptores de memoria diferentes, uno para código y otro para datos 17

6.2 Segmento de datos de solo lectura 18

6.3 Registros de segmento en modo protegido 19

7 **Conclusión** **20**

1 Introducción

En el presente trabajo se busca analizar y experimentar con distintas etapas del proceso de arranque de una computadora, poniendo el foco en cómo un programa muy pequeño puede ser cargado por el firmware y tomar control de la ejecución desde los primeros instantes del encendido. Para ello se parte de la construcción de imágenes booteables mínimas en Assembly, con el objetivo de comprender el formato del MBR, la función del BIOS y la forma en que el código es ubicado y ejecutado en memoria.

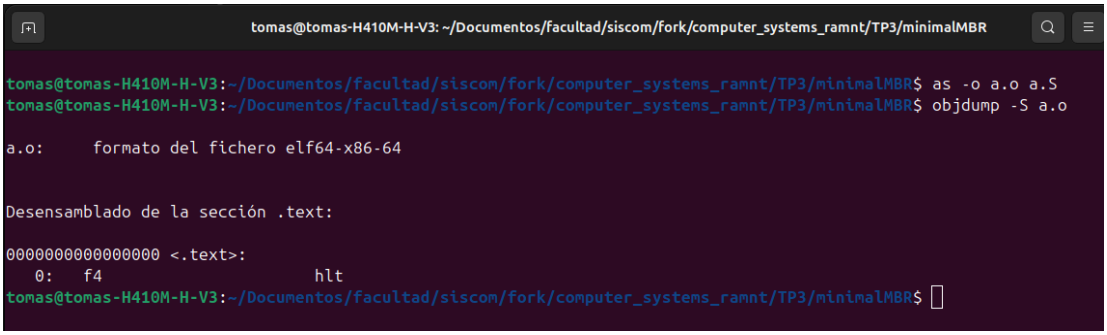
Además del desarrollo práctico, el informe pretende relacionar estas experiencias con conceptos más amplios del arranque moderno, incluyendo el rol de UEFI, coreboot y otros componentes del firmware. También se estudia la importancia del linker en la generación de imágenes binarias, el uso de herramientas como `objdump`, `hd` y GDB para inspeccionar y depurar el código, y finalmente la transición desde modo real hacia modo protegido mediante una GDT mínima. En conjunto, el propósito del trabajo es integrar teoría y práctica para comprender con mayor claridad cómo se organiza la secuencia de boot a bajo nivel.

2 MBR mínimo

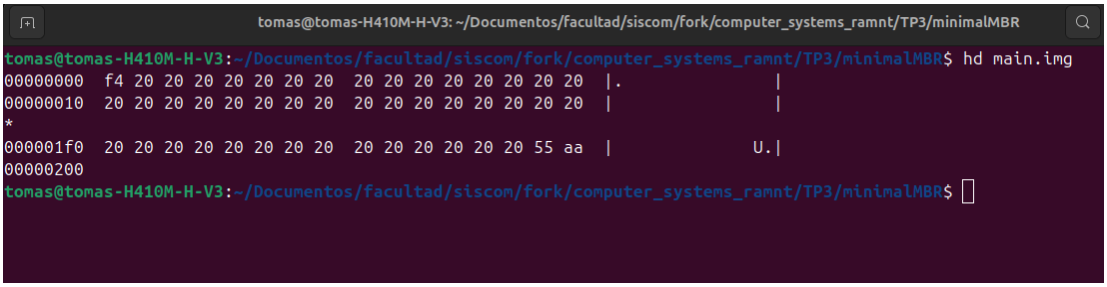
En esta primera prueba se construyó un **MBR mínimo** cuyo único código de arranque es la instrucción `hlt`. El archivo [a.S] contiene solamente esa instrucción, por lo que el objetivo no fue mostrar funcionalidad visible, sino verificar el formato mínimo de una imagen booteable.

2.1 Construcción del MBR

Primero se ensambló el archivo y se comprobó con `objdump` que el código generado corresponde efectivamente a `hlt`.

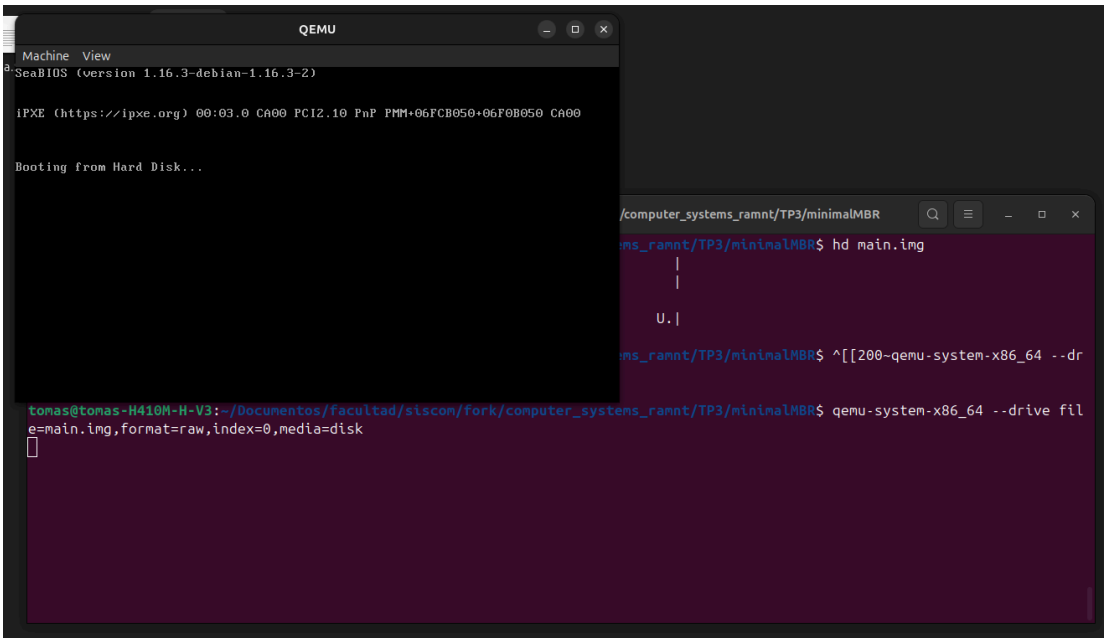


Luego se generó `main.img` con 512 bytes: el primer byte contiene `0xF4` (`hlt`) y al final del sector se colocó la firma de arranque `0x55AA`, que es la que reconoce el BIOS para considerar válido el MBR.

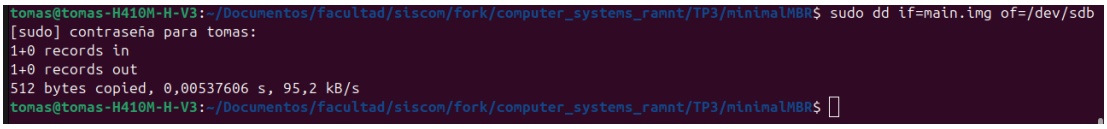


2.2 Prueba de arranque

La imagen resultante se ejecutó con QEMU como disco booteable. El equipo virtual intenta arrancar desde el disco y, al ejecutar `hlt`, la secuencia se detiene inmediatamente, lo que confirma que el control llegó al código del MBR.



Como prueba adicional, la imagen también se copió a un pendrive con `dd`, lo que muestra que el archivo generado puede escribirse directamente como sector de arranque.



3 UEFI y Coreboot

3.1 UEFI

UEFI, sigla de **Unified Extensible Firmware Interface**, es una interfaz de firmware que reemplaza al BIOS tradicional en la mayoría de los equipos modernos. Su función es inicializar el hardware al encender la máquina, preparar el entorno mínimo de ejecución y transferir el control al cargador de arranque o al sistema operativo.

A diferencia del BIOS clásico, UEFI ofrece una arquitectura más modular, soporte para discos grandes mediante GPT, posibilidad de trabajar en 32 o 64 bits, controladores propios, variables persistentes en memoria no volátil y un modelo de servicios accesible mediante interfaces bien definidas.

3.2 Uso de UEFI

UEFI puede utilizarse de distintas maneras según el nivel de desarrollo que se quiera abordar. Desde la perspectiva de un usuario, se lo usa cuando el equipo arranca un sistema operativo en modo UEFI. Desde la perspectiva de un desarrollador de bajo nivel, puede interactuarse con UEFI escribiendo una aplicación EFI, es decir, un ejecutable con formato PE que el firmware carga y ejecuta antes de iniciar el sistema operativo.

Esa interacción se realiza a través de tablas y punteros a servicios que el firmware expone. Por ejemplo, una aplicación EFI puede invocar servicios de consola, acceso al sistema de archivos, manejo de memoria o consulta de variables del firmware. Una función que podría llamarse con esta dinámica es `OutputString`, perteneciente al protocolo de salida de texto simple, que permite escribir texto en pantalla durante la etapa de arranque. Esta función recibe una cadena en formato UTF-16 y la muestra por la consola de UEFI, por lo que resulta útil para programas de prueba, menús de arranque o mensajes de diagnóstico tempranos.

3.3 Casos de bugs de UEFI

Existen múltiples antecedentes de vulnerabilidades explotables en implementaciones UEFI. Este tipo de fallas es especialmente sensible porque se ejecuta antes del sistema operativo y con un alto nivel de privilegio.

Un caso conocido es **LogoFAIL**, una familia de vulnerabilidades publicadas en 2023 que afectó analizadores de imágenes usados por distintos fabricantes dentro del firmware UEFI. Mediante imágenes especialmente manipuladas, un atacante podía lograr ejecución de código durante el arranque y evadir mecanismos como Secure Boot.

Otro ejemplo importante es **BootHole**, divulgado en 2020. En ese caso, el problema estaba vinculado con GRUB2 en escenarios de arranque seguro y permitía, bajo ciertas condiciones, ejecutar código no confiable durante el proceso de boot. Aunque no todas estas fallas residen en el núcleo del estándar UEFI, sí impactan directamente en la cadena de arranque moderna basada en firmware UEFI.

3.4 Converged Security and Management Engine (CSME) y Intel MEBx

Converged Security and Management Engine (CSME) es un subsistema integrado en muchas plataformas Intel modernas. Funciona sobre un microcontrolador separado del procesador principal y permanece activo incluso cuando el sistema operativo todavía no arrancó o cuando la máquina está en ciertos estados de bajo consumo.Su propósito es proveer funciones de:

1. Seguridad
2. Gestión remota
3. Arranque confiable
4. Inicialización de plataforma
5. Servicios auxiliares para el firmware.

Dentro de ese ecosistema aparece **Intel Management Engine BIOS Extension (Intel MEBx)**, que es una interfaz de configuración accesible desde el firmware del equipo. MEBx permite ajustar parámetros relacionados con Intel Management Engine, especialmente en entornos corporativos donde se usan capacidades como administración remota, aprovisionamiento o políticas de seguridad.

3.5 Coreboot

Coreboot es un proyecto de firmware libre orientado a reemplazar BIOS o firmware propietario por una etapa de inicialización mínima, rápida y auditable. Su objetivo principal es realizar solo la configuración esencial del hardware y luego transferir el control

a una carga útil, llamada *payload*, como por ejemplo SeaBIOS, GRUB, LinuxBoot o incluso un kernel Linux.

En lugar de incorporar una gran cantidad de lógica cerrada dentro del firmware, coreboot reduce la complejidad visible y deja que componentes posteriores realicen tareas de más alto nivel.

Coreboot aparece en distintos tipos de productos. Uno de los casos más conocidos es el de varios **Chromebooks**, donde Google lo ha utilizado en numerosos dispositivos. También se encuentra en equipos de fabricantes o integradores enfocados en software libre y seguridad, como algunas configuraciones de **System76**, **Purism** y plataformas embebidas o de servidor específicas.

Además, coreboot es frecuente en entornos de laboratorio, investigación, proyectos educativos y hardware personalizado, donde resulta valioso contar con mayor control sobre la secuencia de arranque.

Las principales ventajas del uso de coreboot son las siguientes:

- Reduce el tiempo de arranque al minimizar la lógica ejecutada en firmware.
- Permite una mayor auditabilidad del código en comparación con soluciones completamente propietarias.
- Brinda mayor flexibilidad para elegir la carga útil más adecuada según el objetivo del sistema.
- Facilita escenarios de personalización, investigación y depuración del proceso de inicialización.
- Puede mejorar la seguridad al hacer más visible el comportamiento del firmware y reducir componentes innecesarios, aunque esto depende de la plataforma concreta.

4 Linker

Un **linker** es la herramienta encargada de tomar uno o más archivos objeto y combinarlos para producir un archivo final ejecutable, una biblioteca o, como en este caso, una imagen binaria de arranque. Su tarea principal es ubicar las secciones del programa en memoria, resolver referencias entre símbolos y generar un formato de salida coherente con el destino deseado.

El **linker** no solo une el código ensamblado, sino que además define en qué dirección debe quedar ubicado dentro de la imagen booteable.

4.1 Dirección 0x7c00 y su importancia

En el script del **linker** aparece la dirección `0x7c00`. Esa es la dirección de memoria física en la que el BIOS carga tradicionalmente el sector de arranque cuando encuentra un disco booteable.

Esta dirección es necesaria porque el programa debe quedar enlazado suponiendo que comenzará a ejecutarse desde esa ubicación. Si el **linker** no conoce esa dirección, no puede calcular correctamente las posiciones de etiquetas, mensajes o saltos dentro del código.

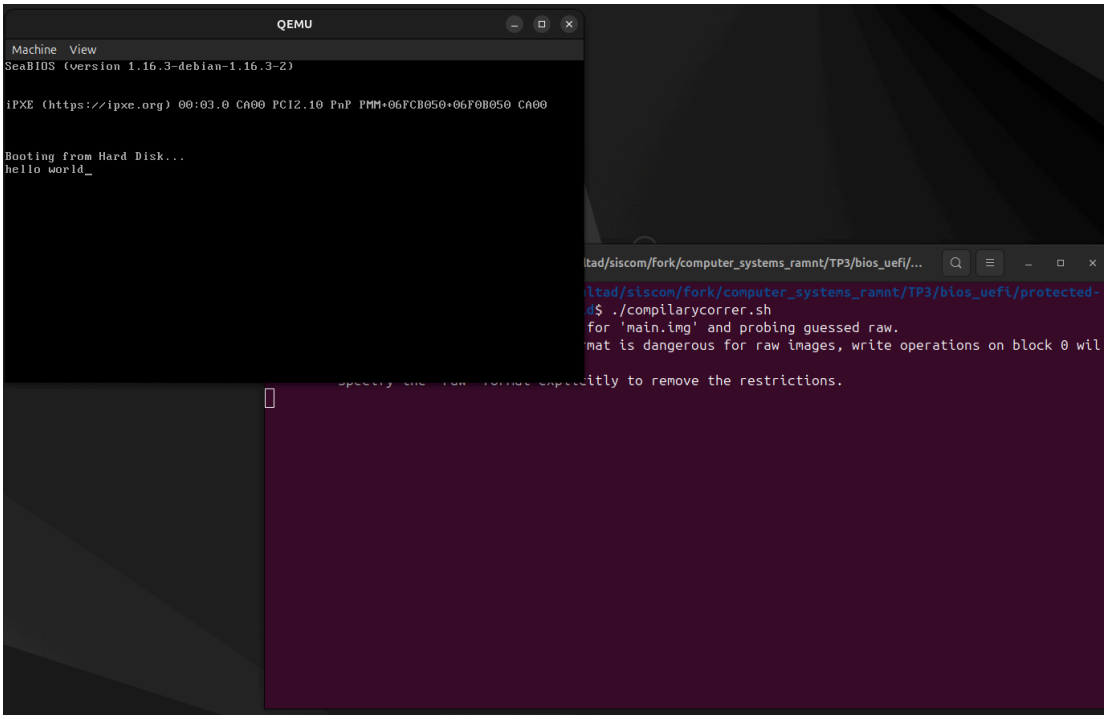
Esto puede verse directamente en el script `link.ld`:

```
SECTIONS
{
    . = 0x7c00;
    .text :
    {
        __start = .;
        *(.text)
        . = 0x1FE;
        SHORT(0xAA55)
    }
}
```

En este fragmento se fija la dirección base `0x7c00`, que es donde el BIOS carga el sector de arranque en memoria, y también se especifica la firma final `0xAA55`, que debe quedar ubicada en los últimos dos bytes del sector.

4.2 Comparativa de objdump y hd

El archivo `compilarycorrer.sh` automatiza el armado y la prueba del ejemplo. Primero ejecuta `as` para generar `main.o`, luego usa `ld` junto con `link.ld` para construir `main.img`, y finalmente corre esa imagen con QEMU. Gracias a eso, con un único script puede compilarse y verificarse el arranque del programa.



Al ejecutar el script, QEMU muestra en pantalla el mensaje `hello world`, lo que confirma que el sector fue cargado correctamente por el BIOS y que el código del bootloader llegó a ejecutarse.

Luego, al comparar las salidas de `objdump -S main.o` con `hd main.img`, se observa el mismo programa desde dos perspectivas distintas: `objdump` muestra el código ensamblado en forma de instrucciones, mientras que `hd` muestra la imagen final byte a byte.

```
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_syst
ected-mode-sdc-master-01HelloWorld/01HelloWorld$ objdump -S main.o

main.o:     formato del fichero elf64-x86-64


Desensamblado de la sección .text:

0000000000000000 <loop-0x5>:
.code16
    mov $msg, %si
   0:   be 00 00 b4 0e          mov    $0xeb40000,%esi

0000000000000005 <loop>:
    mov $0x0e, %ah
loop:
    lodsb
   5:   ac                      lods   %ds:(%rsi),%al
    or %al, %al
   6:   08 c0                   or     %al,%al
    jz halt
   8:   74 04                   je     e <halt>
    int $0x10
   a:   cd 10                   int    $0x10
    jmp loop
   c:   eb f7                   jmp    5 <loop>

000000000000000e <halt>:
halt:
    hlt
   e:   f4                      hlt

000000000000000f <msg>:
   f:   68 65 6c 6c 6f          push   $0x6f6c6c65
  14:   20 77 6f                and    %dh,0x6f(%rdi)
  17:   72 6c                   jb     85 <msg+0x76>
  19:   64                      fs
```

En la salida de `objdump` se ven desplazamientos pequeños dentro del archivo objeto, por ejemplo `0`, `5`, `e` y `f`. Eso ocurre porque `main.o` todavía es un objeto intermedio. La dirección real de carga `0x7c00` no aparece allí como dirección visible de ejecución, ya que `objdump` muestra offsets relativos dentro del objeto y no la ubicación final en memoria.

```
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/
ected-mode-sdc-master-01HelloWorld/01HelloWorld$ hd main.img
00000000  be 0f 7c b4 0e ac 08 c0  74 04 cd 10 eb f7 f4 68  |..|.....t......h|
00000010  65 6c 6c 6f 20 77 6f 72  6c 64 00 66 2e 0f 1f 84  |ello world.f....|
00000020  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000030  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000040  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
00000050  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
00000060  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
00000070  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000080  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000090  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
000000a0  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
000000b0  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
000000c0  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
000000d0  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
000000e0  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
000000f0  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
00000100  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
00000110  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000120  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000130  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
00000140  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
00000150  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
00000160  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000170  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000180  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
00000190  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
000001a0  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
000001b0  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
000001c0  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
000001d0  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
000001e0  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
000001f0  00 66 2e 0f 1f 84 00 00  00 00 00 0f 1f 00 55 aa  |.f............U.|
00000200
```

En `hd`, en cambio, puede verse la imagen final completa. Los primeros bytes del archivo corresponden al código máquina del programa, y enseguida aparece la cadena `hello world` en la columna ASCII. Esto indica que el programa fue colocado al comienzo del sector booteable. Sin embargo, tampoco aquí se ve de forma directa la dirección `0x7c00`, porque `hd` muestra el archivo en disco y no la memoria RAM. Lo que sí puede verificarse es que ese contenido será el que el BIOS copie luego a memoria en la dirección indicada por el linker.

También se observa al final de la imagen la firma `55 aa`, ubicada en el offset `0x1FE`. Esa es la representación en bytes de la firma de arranque definida en el linker mediante `SHORT(0xAA55)`. En otras palabras, en el script se expresa como `0xAA55`, pero en el volcado hexadecimal se ve almacenada como `55 aa` en los últimos dos bytes del sector.

Por lo tanto, `objdump` y `hd` permiten verificar el contenido del programa y su ubicación dentro de la imagen, mientras que la dirección de carga `0x7c00` se determina a partir del script del linker y del comportamiento estándar del BIOS.

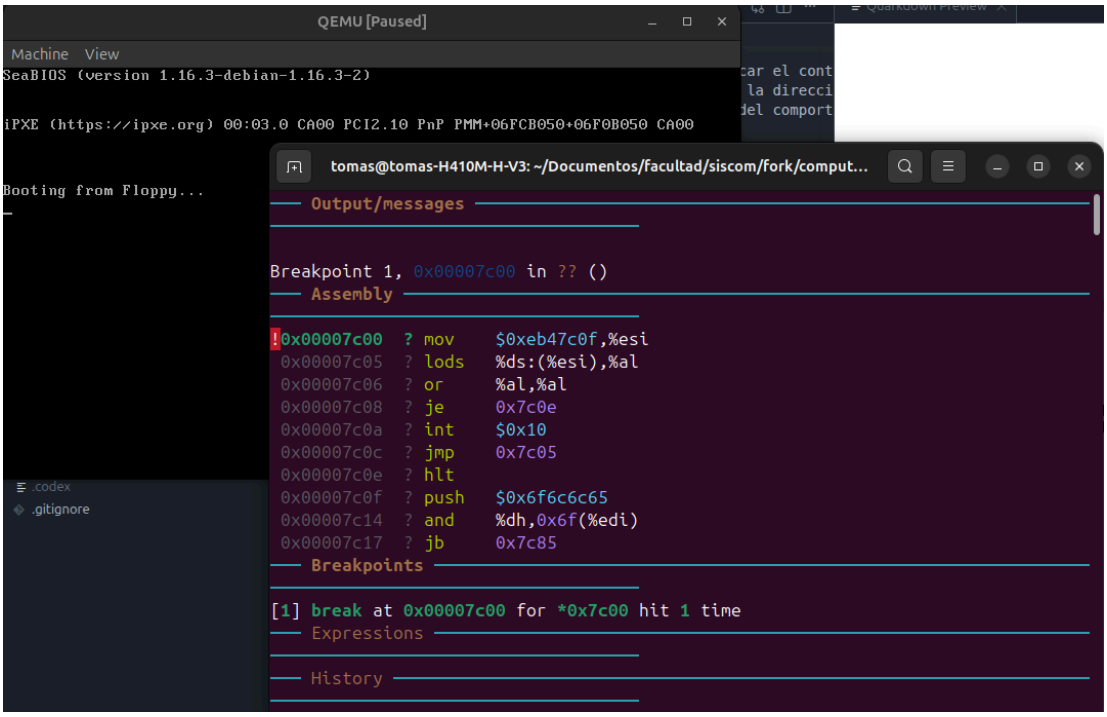
4.3 Opción `--oformat binary` en el linker

La opción **`--oformat binary`** se utiliza para que el linker genere una salida en **binario plano**, sin encabezados ELF ni metadatos adicionales. Esto es fundamental en un MBR, porque el BIOS no espera un archivo ejecutable con estructura compleja, sino un sector crudo de 512 bytes que pueda copiar a memoria y ejecutar directamente.

Sin esta opción, el resultado del enlazado sería un archivo con formato de objeto o ejecutable estándar, que no serviría como sector de arranque.

5 Depuración con GDB Dashboard

Para depurar el programa se ejecutó QEMU con las opciones `-s -S`, lo que permite conectar GDB antes de que el bootloader comience a correr. Una vez hecha la conexión, GDB Dashboard muestra el estado de la CPU y el listado de instrucciones cargadas en memoria.



En esta captura puede verse de manera directa que la ejecución comienza en la dirección `0x7c00`. A diferencia de `objdump` y `hd`, aquí sí se observa la dirección real de memoria en tiempo de ejecución, ya que GDB está inspeccionando la máquina virtual ya cargada por el BIOS.

Luego se colocaron dos breakpoints manuales en las direcciones `0x7c08` y `0x7c0c`, que corresponden a las instrucciones `jz halt` y `jmp loop`. La idea fue detener la ejecución justo antes y justo después de la interrupción `int $0x10`, para observar cómo el programa recorre la cadena carácter por carácter.

```
>>> break *0x7c08
Punto de interrupción 2 at 0x7c08
>>>
break *0x7c0c
orden indefinida: «». Intente con «help»
>>> break *0x7c0c
Punto de interrupción 3 at 0x7c0c
>>>
```

Al continuar la ejecución, el primer corte permite ver que ya se imprimieron los primeros caracteres de la cadena y que el flujo volvió al punto de decisión del bucle.

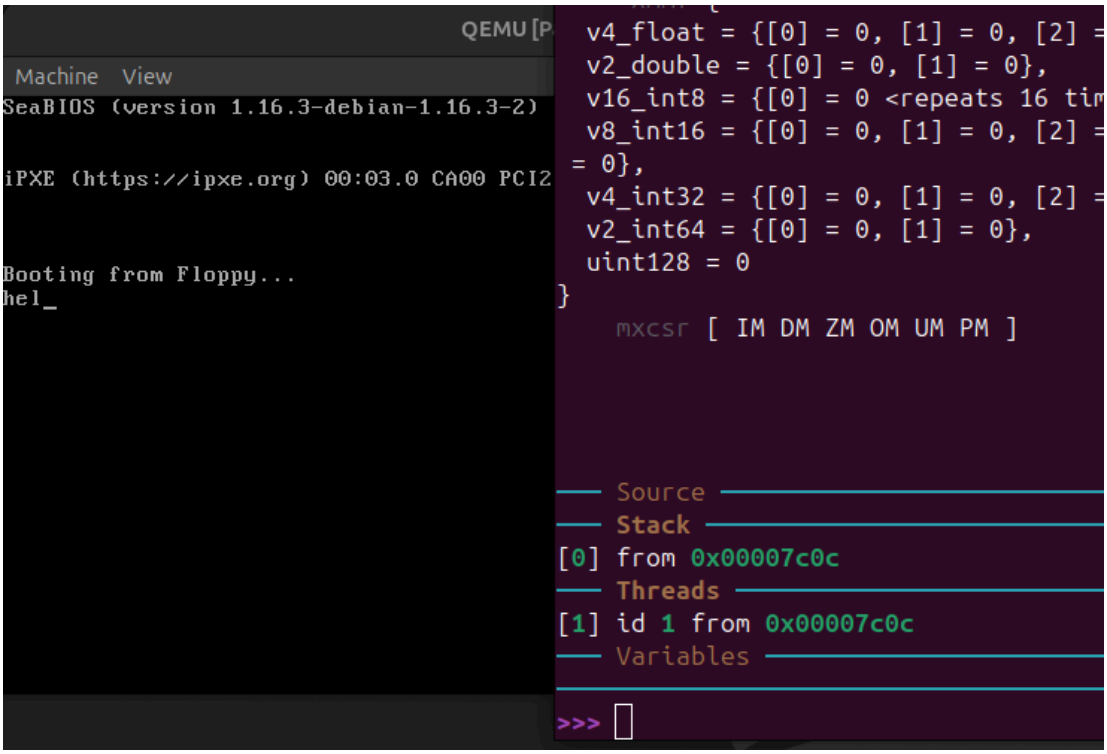
```
Booting from Floppy...
he_
```

```
  uint128 = 0
}
    mxcsr [ IM DM ZM OM UM PM ]

── Source
── Stack
[0] from 0x00007c08
── Threads
[1] id 1 from 0x00007c08
── Variables

>>> c
```

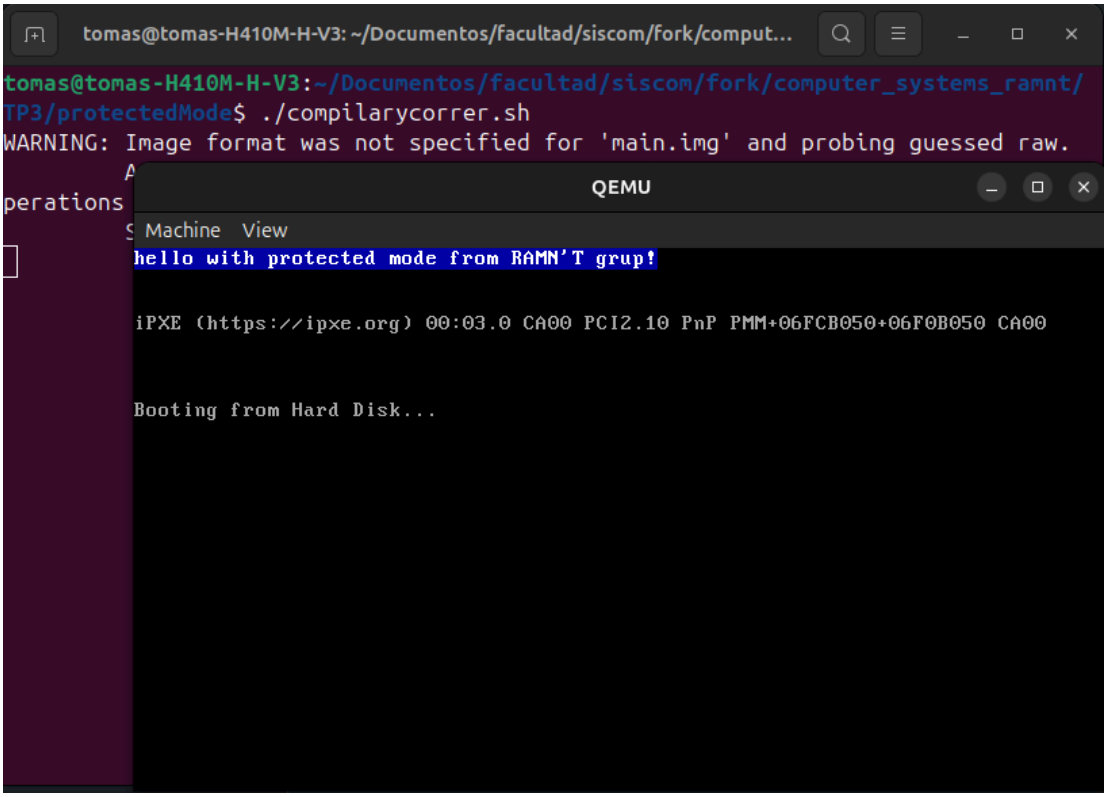
En la siguiente continuación se observa que la impresión avanza un carácter más. Esto confirma que en cada iteración se carga un byte de la cadena, se ejecuta la interrupción de video y luego se vuelve al inicio del bucle hasta encontrar el byte nulo final.


QEMU [P
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2
Booting from Floppy...
hel_
v4_float = {[0] = 0, [1] = 0, [2] =
v2_double = {[0] = 0, [1] = 0},
v16_int8 = {[0] = 0 <repeats 16 tim
v8_int16 = {[0] = 0, [1] = 0, [2] =
= 0},
v4_int32 = {[0] = 0, [1] = 0, [2] =
v2_int64 = {[0] = 0, [1] = 0},
uint128 = 0
}
mxcsr [IM DM ZM OM UM PM]
Source
Stack
[0] from 0x00007c0c
Threads
[1] id 1 from 0x00007c0c
Variables
>>>

6 Modo protegido

En esta etapa se creó un nuevo archivo en Assembly para ejecutar nuevamente el programa de impresión, pero esta vez realizando antes la transición del procesador desde modo real a **modo protegido**. Para eso se definió una GDT mínima, se cargó con `lgdt`, se activó el bit PE de `CR0` y finalmente se hizo un salto largo al segmento de código de 32 bits.

Una vez en modo protegido, el programa ya no utiliza la interrupción BIOS `int 0x10` para mostrar texto, sino que escribe directamente en la memoria de video VGA en la dirección `0xb8000`. De ese modo se mantiene la funcionalidad de impresión, pero usando un mecanismo compatible con el nuevo modo de ejecución.



La captura muestra que el programa logró entrar en modo protegido y escribir el mensaje en pantalla correctamente, cumpliendo el mismo objetivo que en la etapa anterior, pero ahora con segmentación y ejecución de 32 bits.

6.1 Programa con dos descriptores de memoria diferentes, uno para código y otro

para datos

En este cado, el programa define en su GDT descriptores con **base** y **límite** distintos para cada segmento. En lugar de tener código y datos comenzando ambos en base `0x00000000` y cubriendo toda la memoria, el descriptor de código podría apuntar, por ejemplo, a una región donde solo reside el código ejecutable, mientras que el descriptor de datos podría apuntar a otra región separada para variables, buffers o pila.

En ese esquema, el procesador traduce las direcciones lógicas usando el selector cargado en cada registro de segmento. Así, `CS` referenciaría el espacio de código y `DS` o `SS` el espacio de datos. La ventaja es que permite separar responsabilidades, limitar accesos y detectar errores cuando un acceso intenta salir de los límites definidos para un segmento.

6.2 Segmento de datos de solo lectura

Si se modifican los bits de acceso del descriptor de datos para que el segmento no admita escritura, una instrucción que intente almacenar allí debería generar una **excepción de protección general**. Es decir, el procesador detecta que el tipo de acceso no está permitido para ese descriptor y aborta la operación.

Para realizar esta experiencia se aprovechó la línea comentada dentro de `main.S`, en la definición de `gdt_data`, donde puede cambiarse el byte de acceso del descriptor. En la versión normal del programa se usa:

```
.byte 0b10010010
```

mientras que para la prueba se reemplazó temporalmente por la línea comentada:

```
#.byte 0b10010000
```

De esa manera, el descriptor de datos quedó configurado con permisos más restrictivos, de modo que al intentar escribir en ese segmento el procesador debía detectar una violación de protección.

En la ejecución práctica con QEMU, el resultado observado fue que el emulador quedó en una especie de reinicio constante. Esto es consistente con el comportamiento esperado para un programa mínimo que no tiene una IDT completa ni manejadores de excepciones instalados: al producirse la falta de protección, el procesador no puede atenderla

correctamente, deriva en una falla más grave y la máquina virtual termina reiniciándose repetidamente.

6.3 Registros de segmento en modo protegido

En modo protegido los registros de segmento no se cargan con direcciones físicas, sino con **selectores**. En este trabajo se usa `0x08` para el segmento de código y `0x10` para el segmento de datos. Esos valores no son arbitrarios: apuntan a entradas concretas dentro de la GDT.

El selector `0x08` referencia el segundo descriptor de la tabla, que es el de código, mientras que `0x10` referencia el tercero, que es el de datos. Esto ocurre porque cada entrada de la GDT ocupa 8 bytes y el índice del descriptor se codifica dentro del selector. Por eso, al pasar a modo protegido, los registros `CS`, `DS`, `ES`, `FS`, `GS` y `SS` deben cargarse con selectores válidos y no con una dirección lineal como sucedía conceptualmente en modo real.

7 Conclusión

El desarrollo de este trabajo permitió recorrer de forma práctica distintas etapas del proceso de arranque de una computadora, desde la construcción de un MBR mínimo hasta la ejecución de código en modo protegido. A lo largo de las pruebas se verificó cómo intervienen el BIOS, el linker, el formato de la imagen booteable y la organización de memoria para que un programa muy pequeño pueda tomar el control de la máquina desde el encendido.

Además, la comparación entre mecanismos tradicionales y modernos de arranque, como BIOS, UEFI y coreboot, ayudó a ubicar cada experiencia dentro de un contexto más amplio. En conjunto, el trabajo sirvió para comprender mejor la relación entre hardware, firmware y software de bajo nivel, y para mostrar que conceptos como sector de arranque, segmentación, GDT y cambio de modo no son solo teóricos, sino elementos concretos que pueden observarse y validarse mediante herramientas reales.